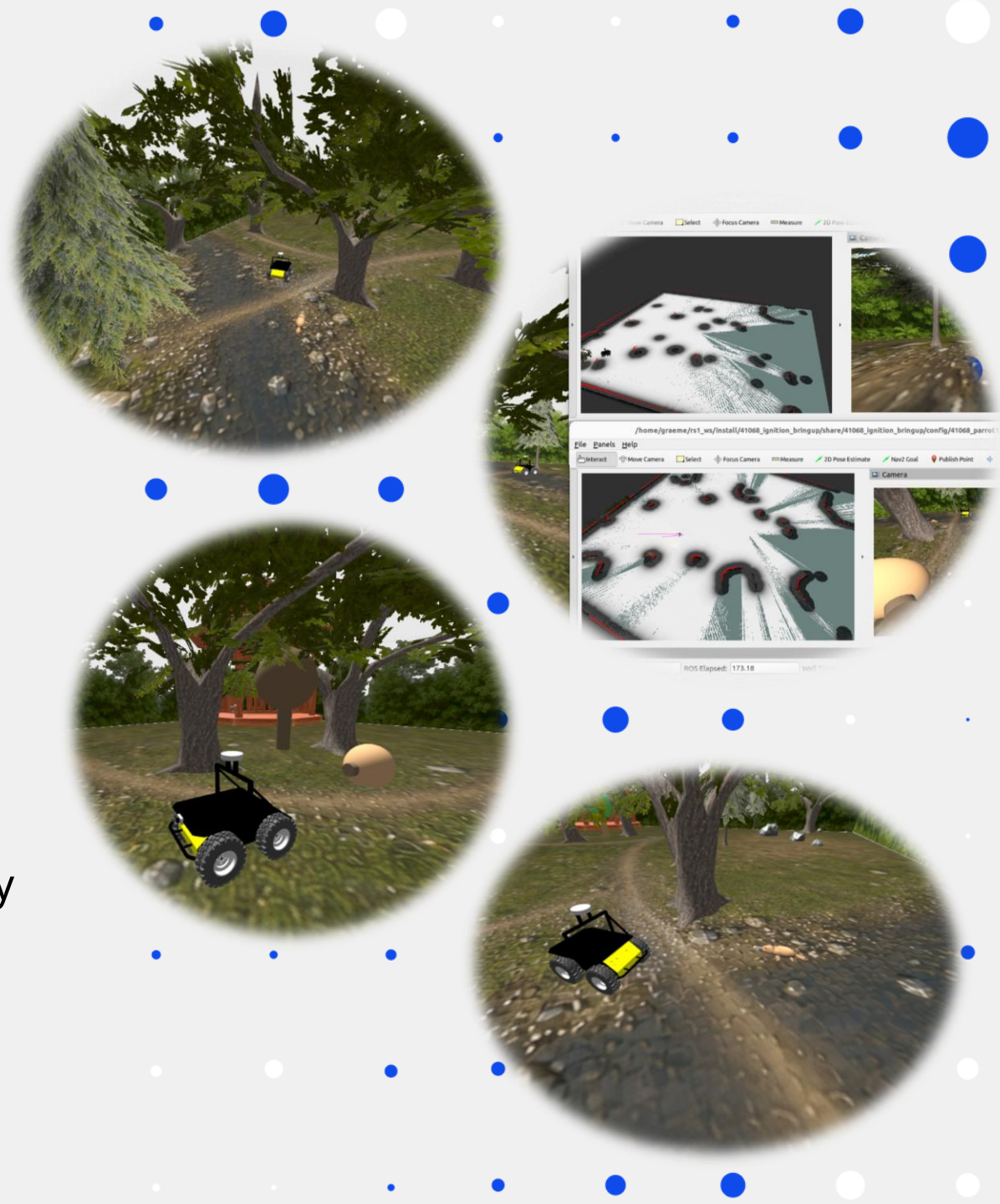




Simulation Package Overview

Graeme Best
41068 Robotics Studio 1

Faculty of Engineering and Information Technology
University of Technology Sydney



The 41068 Simulation Starter Package



- ROS 2 package
 - Gazebo world of a forest environment
 - Ground and aerial robots with sensors
 - Simple autonomy stack
 - Example autonomy code
 - Example dynamic world features
- Just enough to get started!
- You should adapt the simulation to meet the needs of your project:
 - Expanded world
 - New objects
 - Modified robots
 - ...



Questions for your 41068 Project



- How will you adapt the simulation environment to support your project?
- How will you adapt the robots and sensors?
- How will you adapt and develop the autonomy?
- How will you test and evaluate your system?



Outline



- Very brief ROS 2 refresher
- Relationship between ROS, Gazebo and RViz
- Package structure
 - Launch files
 - Namespaces
 - Gazebo-ROS bridges
 - Config files
 - Robot descriptions
 - Worlds and models
 - Supplied code
- Ideas for adapting the simulation



ROS and Gazebo refresher

What is the Robot Operating System (ROS)?



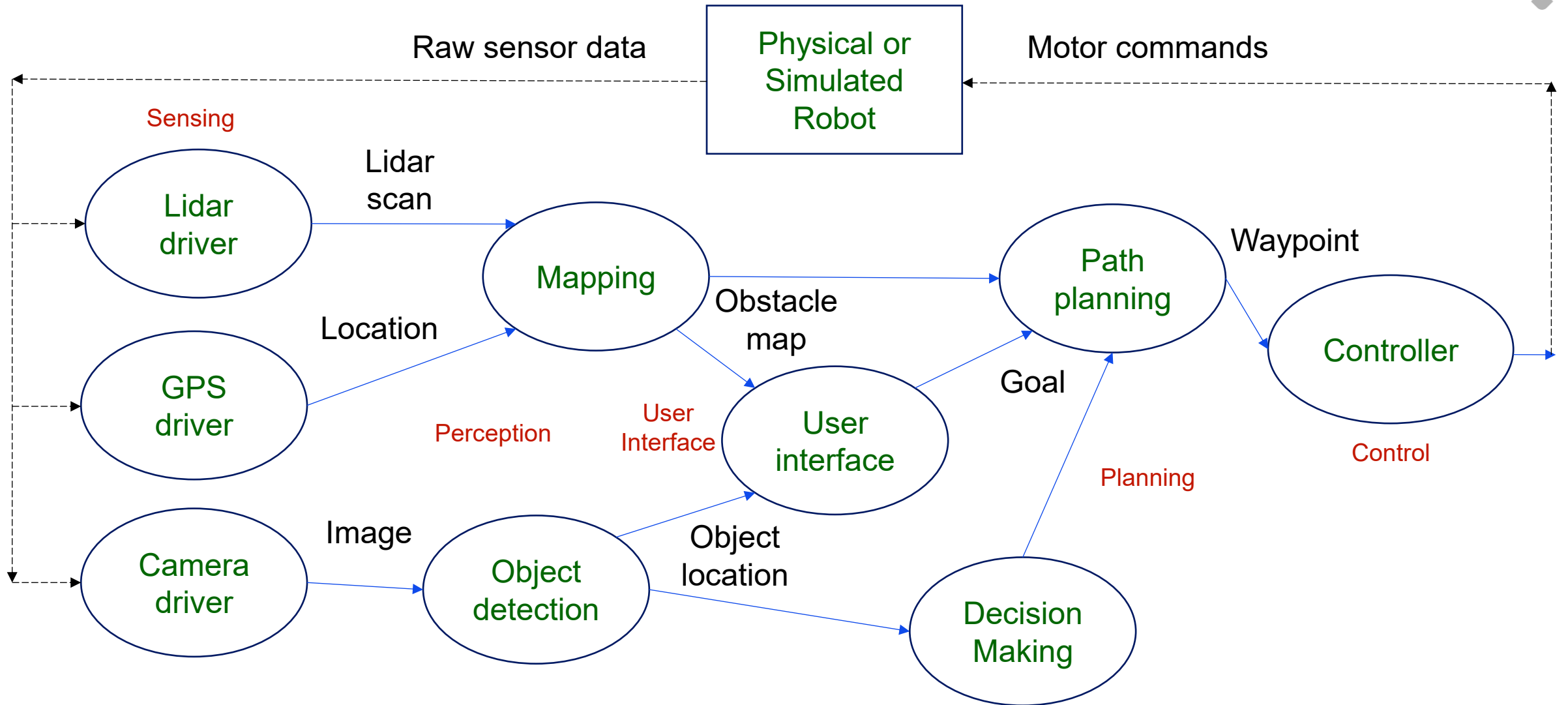
- ROS 2 is a collection of software libraries, communication tools and conventions for developing robot systems
- A ROS system is normally composed of many separate software components
- These components exchange data rather than being combined into one large program
- ROS provides interfaces and communication

Key ROS Concepts



- **Nodes**: running software components
- **Topics**: continuous streams of data
- **Messages**: the structure of data sent through topics
- **Parameters**: the values used to configure nodes
- **Services**: request-and-response interactions
- **Actions**: longer-running goals with feedback and results
- **TF**: coordinate frames and transformations
- **Launch files**: start and configure collections of nodes

Relatively Simple Example



ROS, Gazebo, and RViz



Ignition Gazebo

- Simulates:
- The environment
 - Robot geometry
 - Collision and physics
 - Joints and actuators
 - Cameras, lidar and IMUs

ROS

- Provides:
- Communication between software components
 - Topics, actions, services, and parameters
 - Autonomy: Mapping, localisation, planning, control, [your code], ...

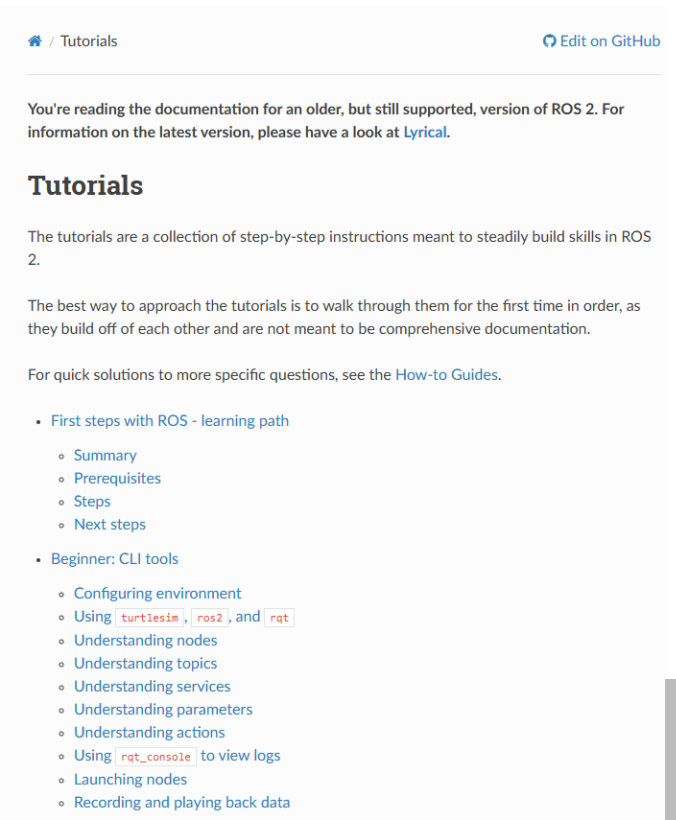
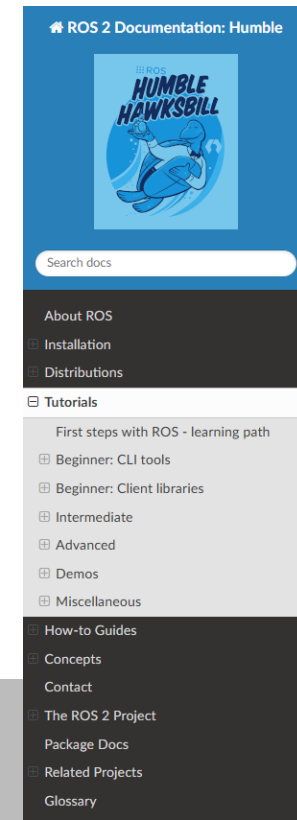
RViz

- Visualises:
- Robot models
 - Sensor data
 - Maps
 - Paths
 - TF frames
 - Navigation goals
 - ...

ROS Learning Resources



- This seminar is not a replacement for the ROS tutorials
- The best way to learn is to try it yourself
 - Use the 41068 package README and working examples
- Revisit 41012 PFMS and related subjects
- Official ROS 2 tutorials
 - <https://docs.ros.org/en/humble/Tutorials.html>





Understanding the Supplied Package

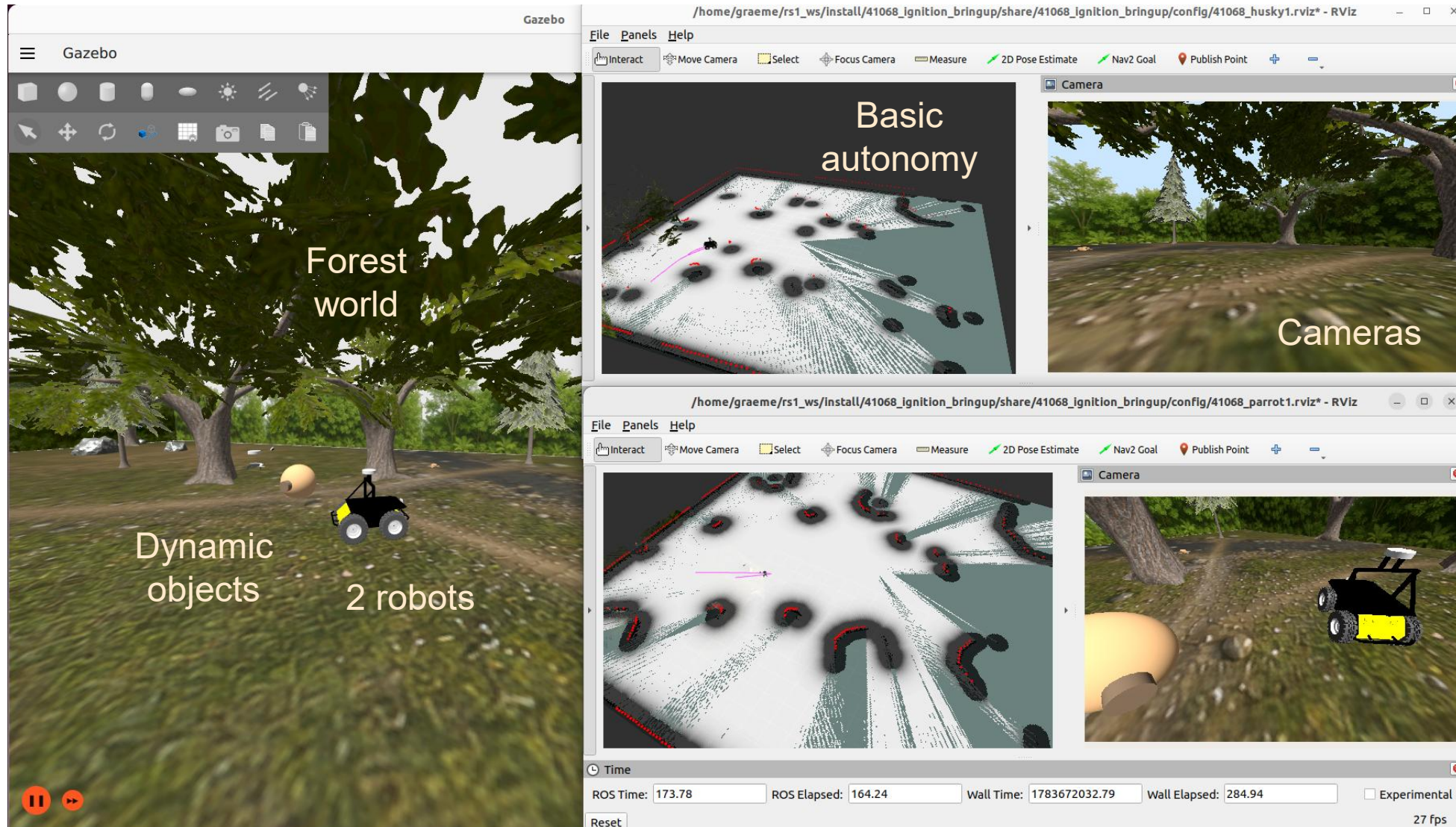
What You Should See



RViz
Visualisation:
UGV

RViz
Visualisation:
UAV

Gazebo
simulator



Package Structure



41068_ignition_bringup/

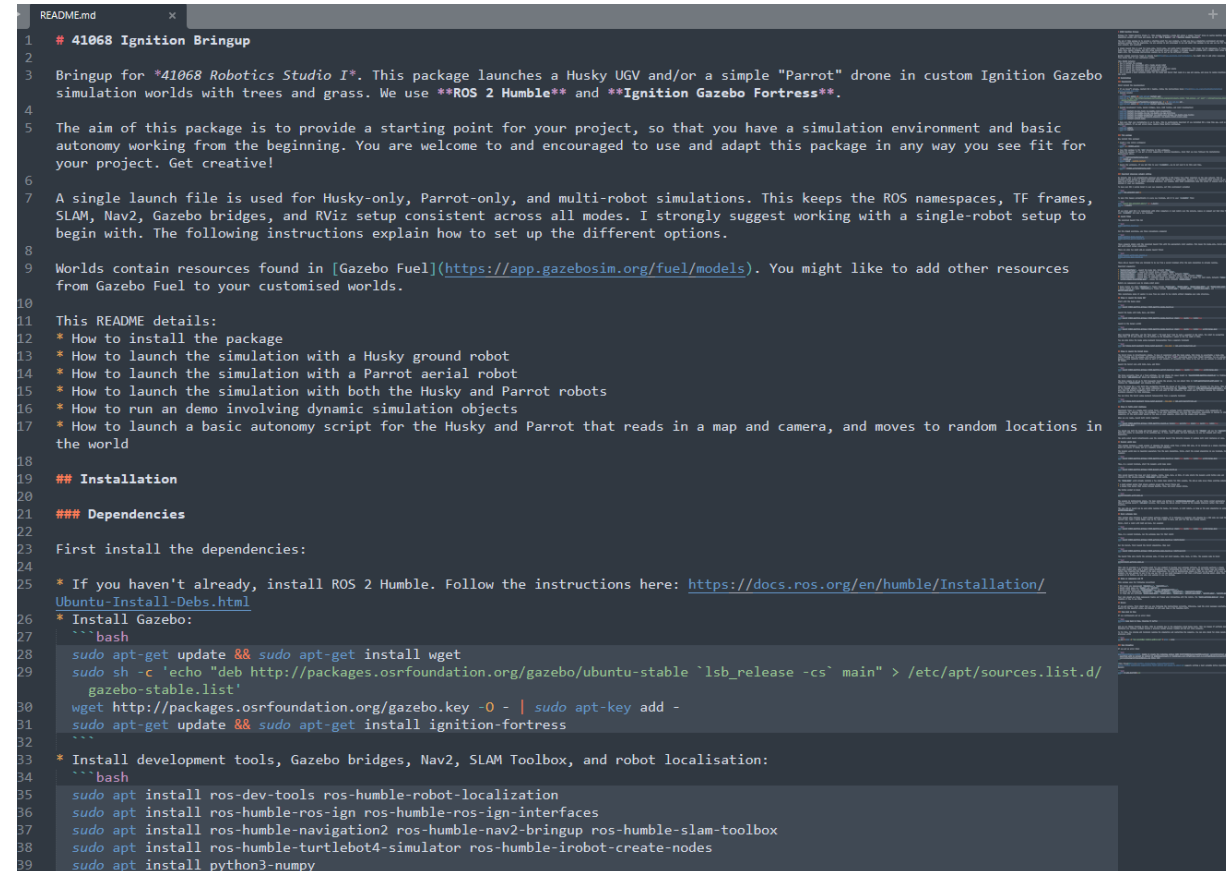
└─ config/	configuration and parameter files
└─ launch/	launch files: start different combinations of the simulation
└─ models/	custom reusable Gazebo environment models
└─ scripts/	two ROS Python examples
└─ urdf_husky/	Husky UGV robot, sensors and Gazebo plugins
└─ urdf_parrot/	Parrot UAV robot, sensors and Gazebo plugins
└─ worlds/	Gazebo world files
└─ CMakeLists.txt	installs the package resources and executable scripts
└─ package.xml	dependencies and package metadata
└─ README.md	installation and launch instructions and tips

Where to Get Started: README.md



➤ Follow the README installation instructions

- Build the workspace
- Launch a simpler configuration first
- Confirm it runs without error
- Drive the robot around
- Try the more complicated configurations
- Preserve this state in Git
- Start making changes



```
1 # 41068 Ignition Bringup
2
3 Bringup for *41068 Robotics Studio I*. This package launches a Husky UGV and/or a simple "Parrot" drone in custom Ignition Gazebo
simulation worlds with trees and grass. We use **ROS 2 Humble** and **Ignition Gazebo Fortress**.
4
5 The aim of this package is to provide a starting point for your project, so that you have a simulation environment and basic
autonomy working from the beginning. You are welcome to and encouraged to use and adapt this package in any way you see fit for
your project. Get creative!
6
7 A single launch file is used for Husky-only, Parrot-only, and multi-robot simulations. This keeps the ROS namespaces, TF frames,
SLAM, Nav2, Gazebo bridges, and RViz setup consistent across all modes. I strongly suggest working with a single-robot setup to
begin with. The following instructions explain how to set up the different options.
8
9 Worlds contain resources found in [Gazebo Fuel](https://app.gazebosim.org/fuel/models). You might like to add other resources
from Gazebo Fuel to your customised worlds.
10
11 This README details:
12 * How to install the package
13 * How to launch the simulation with a Husky ground robot
14 * How to launch the simulation with a Parrot aerial robot
15 * How to launch the simulation with both the Husky and Parrot robots
16 * How to run an demo involving dynamic simulation objects
17 * How to launch a basic autonomy script for the Husky and Parrot that reads in a map and camera, and moves to random locations in
the world
18
19 ## Installation
20
21 ### Dependencies
22
23 First install the dependencies:
24
25 * If you haven't already, install ROS 2 Humble. Follow the instructions here: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html
26 * Install Gazebo:
27   ```bash
28   sudo apt-get update && sudo apt-get install wget
29   sudo sh -c 'echo "deb http://packages.osrfoundation.org/gazebo/ubuntu-stable `lsb_release -cs` main" > /etc/apt/sources.list.d/
gazebo-stable.list'
30   wget http://packages.osrfoundation.org/gazebo.key -O - | sudo apt-key add -
31   sudo apt-get update && sudo apt-get install ignition-fortress
32   ```
33 * Install development tools, Gazebo bridges, Nav2, SLAM Toolbox, and robot localisation:
34   ```bash
35   sudo apt install ros-dev-tools ros-humble-robot-localization
36   sudo apt install ros-humble-ros-ign ros-humble-ros-ign-interfaces
37   sudo apt install ros-humble-navigation2 ros-humble-nav2-bringup ros-humble-slam-toolbox
38   sudo apt install ros-humble-turtlebot4-simulator ros-humble-irobot-create-nodes
39   sudo apt install python3-numpy
```


Launch Files



- The canonical launch file is: `41068_ignition.launch.py`
- It supports these main options:
 - `husky:=true/false`
 - `parrot:=true/false`
 - `slam:=true/false`
 - `nav2:=true/false`
 - `rviz:=true/false`
 - `world:=simple_trees/large_demo`
- `41068_ignition_husky.launch.py` calls the main launch file with Husky only
- `41068_ignition_parrot.launch.py` calls the main launch file with Parrot only

What the Main Launch File Starts



- The canonical launch file performs the following sequence:
 1. Starts the Gazebo world and bridges the simulation clock
 2. Generates and spawns each enabled robot
 3. Starts robot state publishing, localisation, and the robot-specific Gazebo bridge
 4. Optionally starts namespaced SLAM and Nav2 instances for each robot
 5. Optionally starts one namespaced RViz process for each robot

Other Launch Files



- `41068_navigation.launch.py`
 - Starts namespaced SLAM and Nav2
 - Helper launch file – shouldn't usually be called directly

- `41068_dynamic_world_demo.launch.py`
 - Starts only the dynamic-world node (described later)
 - Assumes the main launch file is already running

- `41068_autonomy_demo.launch.py`
 - Starts only the example autonomy node (described later)
 - Assumes the main launch file is already running

Namespaces and TF Frames



- The supplied robots are always *namespaced*, even when running only one robot
- Namespaces allow separate copies of nodes/topics for each robot
- Multiple robots are an option – start by getting one robot working first!

Husky

```
ROS namespace:    /husky1
ROS topics:       /husky1/scan
                  /husky1/camera/image
                  /husky1/cmd_vel
TF frame prefix:  husky1_
TF frames:        husky1_map
                  husky1_odom
                  husky1_camera_link
Gazebo model:     husky1
```

Parrot

```
ROS namespace:    /parrot1
ROS topics:       /parrot1/scan
                  /parrot1/camera/image
                  /parrot1/cmd_vel
TF frame prefix:  parrot1_
TF frames:        parrot1_map
                  parrot1_odom
                  parrot1_camera_link
Gazebo model:     parrot1
```

Config Files

- Gazebo-ROS bridges
 - Global clock
 - Husky/Parrot topics
- Robot localisation
 - Parameters for the localisation node
- SLAM toolbox
 - Parameters for the SLAM node
- Nav2
 - Parameters for navigation, planning, and control
- Rviz
 - RViz visualisation configuration
 - Best edited directly in rviz then saved

```
controller_server:
  ros__parameters:
    use_sim_time: true
    controller_frequency: 5.0
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.5
    min_theta_velocity_threshold: 0.001
    failure_tolerance: 0.3
    progress_checker_plugin: progress_checker
    goal_checker_plugins:
      - general_goal_checker
    controller_plugins:
      - FollowPath
    progress_checker:
      plugin: nav2_controller::SimpleProgressChecker
      required_movement_radius: 0.5
      movement_time_allowance: 10.0
    general_goal_checker:
      stateful: true
      plugin: nav2_controller::SimpleGoalChecker
      xy_goal_tolerance: 1.0
      yaw_goal_tolerance: 1.57
    FollowPath:
      plugin: dwb_core::DWBLocalPlanner
      debug_trajectory_details: false
      min_vel_x: 0.0
      min_vel_y: 0.0
      max_vel_x: 1.0
      max_vel_y: 0.0
      max_vel_theta: 1.5
      min_speed_xy: 0.0
      max_speed_xy: 1.0
      min_speed_theta: 0.0
      acc_lim_x: 2.5
      acc_lim_y: 0.0
      acc_lim_theta: 3.2
      decel_lim_x: -2.5
      decel_lim_y: 0.0
      decel_lim_theta: -3.2
```

Two Supplied Robots



➤ Husky:

- Four-wheeled ground robot
- Differential-drive control
- Simulated odometry
- IMU
- GPU lidar
- RGB-D camera



➤ Parrot:

- Visually resembles a quadrotor
- Deliberately simplified
- Uses a Husky-like planar movement
- Gravity is disabled
- Operates at a fixed altitude
- Collisions are disabled so it can navigate through leaves



Robot Models

➤ URDF.xacro files describe:

- Links and joints
- Visual and collision geometry
- Inertial properties
- Robot and sensor frames
- Reusable variables and macros

➤ Gazebo.xacro files add:

- Simulated sensors and their parameters
- Gazebo topic and model names
- Drive and controller plugins

```
<gazebo reference="${prefix}base_scan">
  <material>Gazebo/FlatBlack</material>
  <sensor type="gpu_lidar" name="lidar">
    <pose relative_to='${prefix}base_scan'>0 0 0 0 0 0</pose>
    <visualize>$(arg laser_visual)</visualize>
    <topic>model/${gz_model_name}/scan</topic>
    <update_rate>3</update_rate>
    <gz_frame_id>${prefix}base_scan</gz_frame_id>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <resolution>1</resolution>
          <min_angle>0.0</min_angle>
          <max_angle>6.28319</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.2</min>
        <max>40</max>
        <resolution>0.015</resolution>
      </range>
      <noise>
        <type>gaussian</type>
        <mean>0.0</mean>
        <stddev>0.01</stddev>
      </noise>
    </ray>
    <always_on>true</always_on>
  </sensor>
</gazebo>

<gazebo reference="${prefix}camera_link">
  <sensor name="rgbd_camera" type="rgbd_camera">
    <camera>
      <horizontal_fov>2.0944</horizontal_fov>
```

World Files

The package contains two provided world files

- `simple_trees.sdf`
 - Simple grass plane
 - Small number of trees
 - Check the basic simulation works
- `large_demo.sdf`
 - Custom forest ground texture
 - Forest boundary walls
 - Collection of trees, rocks, other objects
 - Dynamic demo objects (see later slide)
 - Richer project starting environment

```
<world name="simple_trees">
  <!-- Basic world setup -->
  <light name="sun" type="directional">
    <cast_shadows>1</cast_shadows>
    <pose>0 0 10 0 -0 0</pose>
    <diffuse>0.8 0.8 0.8 1</diffuse>
    <specular>0.2 0.2 0.2 1</specular>
    <attenuation>
      <range>1000</range>
      <constant>0.9</constant>
      <linear>0.01</linear>
      <quadratic>0.001</quadratic>
    </attenuation>
    <direction>-0.5 0.1 -0.9</direction>
  </light>
  <gravity>0 0 -9.81</gravity>
  <magnetic_field>6e-06 2.3e-05 -4.2e-05</magnetic_field>
  <atmosphere type="adiabatic"/>
  <physics name="default_physics" type="ignored">
    <max_step_size>0.01</max_step_size>
    <real_time_factor>1</real_time_factor>
    <real_time_update_rate>100</real_time_update_rate>
  </physics>
  <scene>
    <ambient>1.0 1.0 1.0</ambient>
    <background>0.8 0.8 0.8</background>
    <shadows>1</shadows>
  </scene>

  <!-- Initialise spherical coordinates somewhere -->
  <spherical_coordinates>
    <latitude_deg>0.0</latitude_deg>
    <longitude_deg>0.0</longitude_deg>
    <elevation>10.0</elevation>
    <heading_deg>0</heading_deg>
    <surface_model>EARTH_WGS84</surface_model>
  </spherical_coordinates>

  <!-- Small ground plane with grass texture -->
  <include>
    <uri>
      model://grass_plane
    </uri>
    <name>grass_plane</name>
    <pose>0 0 0 0 0 0</pose>
  </include>

  <!-- Oak tree -->
  <include>
    <uri>
      https://fuel.gazebosim.org/1.0/OpenRobotics/models/Oak%20tree
    </uri>
    <name>oak1</name>
    <pose>0 3.0 0 0 0 0</pose>
  </include>

  <!-- Pine tree -->
  <include>
    <uri>
      https://fuel.gazebosim.org/1.0/OpenRobotics/models/Pine%20tree
```

Lighting

Physics

Terrain

Trees,
rocks...

Models, Textures, and Gazebo Fuel



- Worlds contain objects from Gazebo Fuel:
<https://app.gazebosim.org/fuel/models>
- The package also contains custom models and textures in the `models` directory
- Supports other formats, like 3D meshes
- You can try adding other Fuel objects or create your own!
- Incrementally add complexity – complex models can significantly slow down the simulation



Provided Script 1: Dynamic World Demo



- It's possible to change parameters of a Gazebo world dynamically from your code!
 - Position, colour, visibility, lighting, ...
- `dynamic_world_demo.py` demonstrates how Python code can alter Gazebo:
 - Moves a small animal marker randomly around the forest
 - Cycles a tree between healthy, fire and burnt appearances
- See the code to find out how this works
- Adapt the code to develop other dynamic features!



Provided Script 2: Basic Autonomy Demo



- `basic_autonomy_demo.py` provides skeleton autonomy code: connects perception, mapping and navigation
- Demonstrates how several ROS interfaces can be connected inside one node
- Is deliberately not intended to be a good perception/planning algorithm
- The node does the following:
 - Subscribes to a map and an RGB image
 - Estimates the brightness of each image
 - Uses TF to find the robot pose in the map
 - Sample waypoint in known free space. Brighter image = distant goal; darker image = closer goal
 - Sends the waypoint to Nav2
 - When goal is reached, chooses another goal



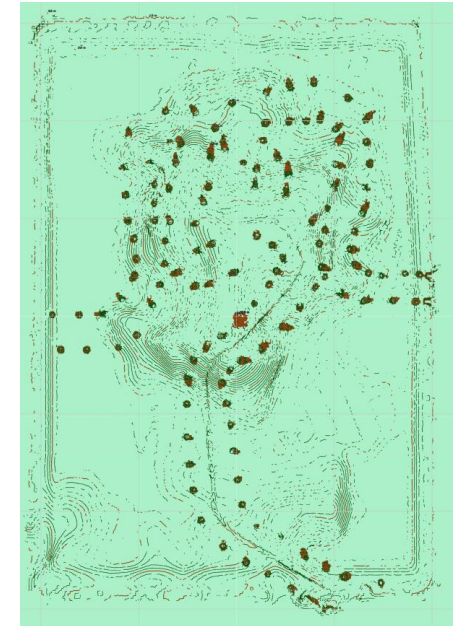
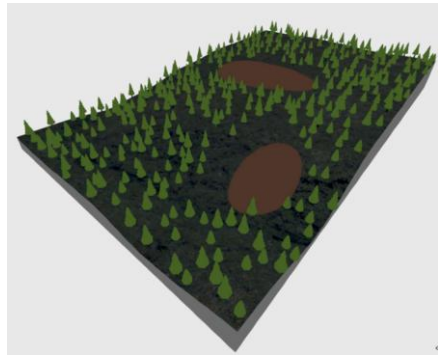
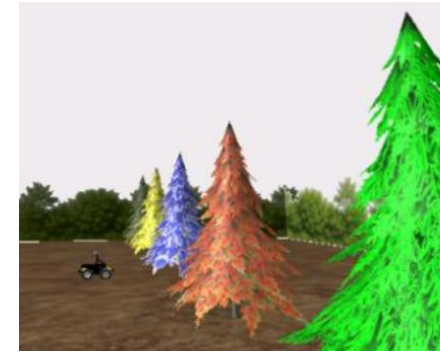
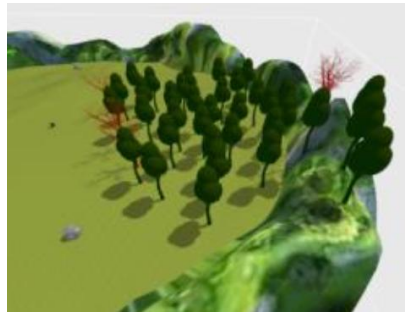
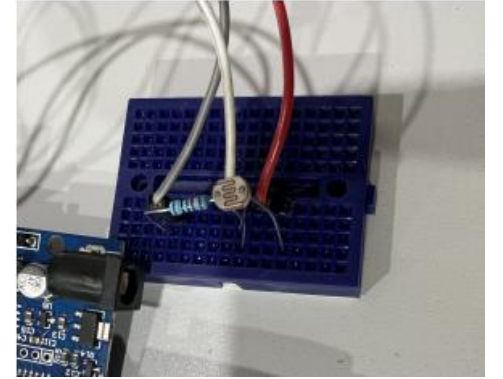
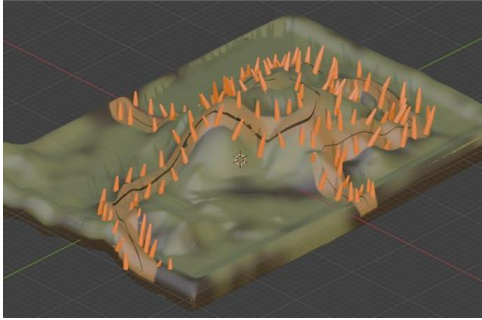
Ideas for Extending the Package

Dimensions You Could Change



- The starter package is only a starting point!
- There are many things you can and should change
 - Environment
 - Dynamic features
 - Robot
 - Sensors
 - Autonomy (perception, decision making, user interface)
 - Multi-robot coordination and communication
 - ...
- Choose changes that support your project. More complexity is not automatically better.

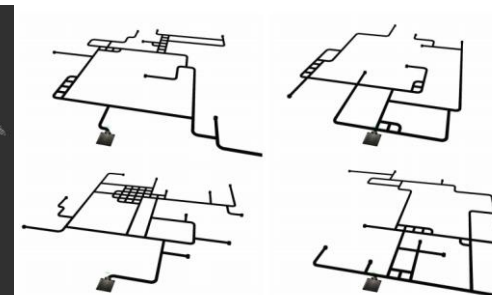
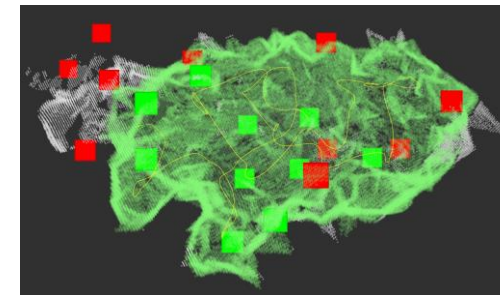
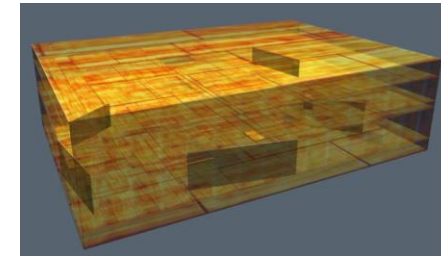
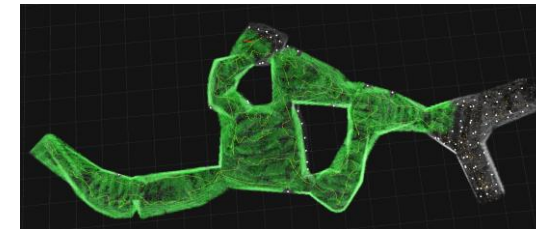
Examples from 41068 in 2025



Procedurally-Generated World Files



- World files follow the “SDF” (Simulation Description Format) format described here: <https://sdformat.org/spec/>
- SDF files are a type of XML file with predefined tags relevant to robotics simulation:
 - World, model, link, joint, pose, visual, collision, material, geometry, sensor, plugin, light, physics, ...
- Modify world files in software:
 - Read in a functioning world file using any **XML parser**
 - Modify/copy/remove XML objects using code
 - Export the modified XML file
 - Open new world file in Gazebo, check functionality
- Get creative – can even create randomised worlds!



Further Resources



- Gazebo Fuel: <https://app.gazebosim.org/fuel/models>
- SDF file format: <https://sdformat.org/spec/>
- An XML parser (there are many others):
<https://docs.python.org/3/library/xml.etree.elementtree.html>
- ROS 2 tutorials: <https://docs.ros.org/en/humble/Tutorials.html>
- Gazebo ignition: <https://gazebosim.org/home>
- Nav2 autonomy: <https://docs.nav2.org/>
- Turtlebot autonomy stack: <https://turtlebot.github.io/turtlebot4-user-manual/>
- 41068 seminars will spark ideas for other technical concepts



Reflection

Reminder of Expectations



- The supplied package gives you a working starting point
- You are not expected to use every supplied component
 - E.g., Start with 1 robot – it is not compulsory to have more than 1 robot
- You should change the simulation in any way relevant to your application:
 - Modify the supplied environment
 - Add/remove robots and sensors
 - Upgrade the autonomy stack
 - Create new ROS nodes
 - Use a completely different simulation setup
- Your goal is to create a simulation and software system that supports your proposed project
- Don't make it unnecessarily complicated, since it will slow down your simulation speed

Questions for your 41068 Project



- How will you adapt the simulation environment to support your project?
- How will you adapt the robots and sensors?
- How will you adapt and develop the autonomy?
- How will you test and evaluate your system?

